

PROGRAM-01

Aim: Implement the quick sort algorithm to sort a given list of elements.

Quick Sort:

- 1.) It is based on divide and conquer approach.*
- 2.) It is based on partition algorithm.*
- 3.) It is an inplace sorting technique and if we are taking extra space that should not be more than $\log(n)$.*
- 4.) It works recursively by first selecting a random pivot value from the list.*
- 5.) At every pass, each element is placed in its proper position.*
- 6.) At each pass, partition algorithm divides the list into elements that are less than the pivot and greater than the pivot.*

PSEUDOCODE:

```
QUICKSORT(A, low, high)
    if low < high then
        pivot ← A[high]
        i ← low - 1

        for j ← low to high - 1 do
            if A[j] ≤ pivot then
                i ← i + 1
                swap A[i] and A[j]

        swap A[i + 1] and A[high]

        pivotIndex ← i + 1

        QUICKSORT(A, low, pivotIndex - 1)
        QUICKSORT(A, pivotIndex + 1, high)
```

CODE-

```
1 #include <stdio.h>
2
3 void quickSort(int arr[], int low, int high) {
4     if (low < high) {
5         int pivot = arr[high];
6         int i = low - 1;
7
8         for (int j = low; j < high; j++) {
9             if (arr[j] < pivot) {
10                 i++;
11                 int temp = arr[i];
12                 arr[i] = arr[j];
13                 arr[j] = temp;
14             }
15         }
16
17         int temp = arr[i + 1];
18         arr[i + 1] = arr[high];
19         arr[high] = temp;
20
21         int pi = i + 1;
22         quickSort(arr, low, pi - 1);
23         quickSort(arr, pi + 1, high);
24     }
25 }
26 void printArray(int arr[], int size) {
27     for (int i = 0; i < size; i++)
28         printf("%d ", arr[i]);
29     printf("\n");
30 }
31 int main() {
32     int n;
33     printf("Enter number of elements: ");
34     scanf("%d", &n);
35
36     int arr[n];
37     printf("Enter %d elements: ", n);
38     for (int i = 0; i < n; i++)
39         scanf("%d", &arr[i]);
40     printf("Original array: ");
41     printArray(arr, n);
42     quickSort(arr, 0, n - 1);
43
44     printf("Sorted array: ");
45     printArray(arr, n);
46
47     return 0;
48 }
```

OUTPUT-

```
Enter number of elements: 4
Enter 4 elements: 10
5
8
20
Original array: 10 5 8 20
Sorted array: 5 8 10 20
```

=== Code Execution Successful ===

Complexity:

Case	Time Complexity	Space Complexity
Best	$O(n \log n)$	$O(\log n)$
Average	$O(n \log n)$	$O(\log n)$
Worst	$O(n^2)$	$O(n)$

PROGRAM-02

Aim: Develop a program to implement merge sort for sorting a list of elements.

Merger Sort:

- 1.) It is a sorting algorithm that is both stable and efficient.*
- 2.) It preserves the relative order of equal elements, ensuring stability in the sorted output.*
- 3.) It follows the divide and conquer approach.*

i.)Divide: Recursively split the input array into two halves until each subset contains only one element.

ii.)Conquer: Recursively sort the two halves.

iii.)Merge: After sorting, merge the two sorted halves back together to form the final sorted array.

PSEUDOCODE:

```
MERGESORT(A, low, high)
    if low < high then
        mid ← (low + high) / 2

        MERGESORT(A, low, mid)
        MERGESORT(A, mid + 1, high)

    create arrays L and R

    n1 ← mid - low + 1
    n2 ← high - mid

    copy A[low...mid] into L[0...n1-1]
    copy A[mid+1...high] into R[0...n2-1]

    i ← 0, j ← 0, k ← low

    while i < n1 AND j < n2 do
        if L[i] ≤ R[j] then
            A[k] ← L[i]
            i ← i + 1
        else
            A[k] ← R[j]
            j ← j + 1
        k ← k + 1
```

```

while i < n1 do
    A[k] ← L[i]
    i ← i + 1
    k ← k + 1

```

```

while j < n2 do
    A[k] ← R[j]
    j ← j + 1
    k ← k + 1

```

CODE-

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void merge(int a[], int l, int m, int r) {
5     int i = 0, j = 0, k = l;
6     int n1 = m - l + 1, n2 = r - m;
7     int L[n1], R[n2];
8
9     for (i = 0; i < n1; i++) L[i] = a[l + i];
10    for (j = 0; j < n2; j++) R[j] = a[m + 1 + j];
11
12    i = j = 0;
13    while (i < n1 && j < n2)
14        a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
15
16    while (i < n1) a[k++] = L[i++];
17    while (j < n2) a[k++] = R[j++];
18 }
19
20 void mergeSort(int a[], int l, int r) {
21     if (l < r) {
22         int m = (l + r) / 2;
23         mergeSort(a, l, m);
24         mergeSort(a, m + 1, r);
25         merge(a, l, m, r);
26     }
27 }
28
29 void printArray(int a[], int n) {
30     for (int i = 0; i < n; i++) printf("%d ", a[i]);
31     printf("\n");
32 }
33
34 int main() {
35     int n;
36     printf("Enter number of elements: ");
37     scanf("%d", &n);
38
39     int arr[n];
40     printf("Enter %d elements: ", n);
41     for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
42
43     printf("Original array: ");
44     printArray(arr, n);
45
46     mergeSort(arr, 0, n - 1);
47
48     printf("Sorted array: ");
49     printArray(arr, n);
50     return 0;
51 }

```

OUTPUT-

```
Enter number of elements: 6
Enter 6 elements: 12 11 13 5 6 7
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
```

```
=== Code Execution Successful ===
```

Complexity:

Case	Time Complexity	Space Complexity
Best	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$

PROGRAM-03

Aim: Implement a solution for the 0/1 knapsack problem to determine the maximum profit achievable within the given weight constraint.

Knapsack problem:

- 1) Knapsack means Bag.*
- 2) The Bag (knapsack) has capacity m .*
- 3) The Object i has weight w_i and corresponding profit is P_i .*
- 4) If abstraction x_i is placed into the knapsack that the profit earned is Px_i .*
- 5) The Objective of this problem is to earn maximum profit.*
- 6) Subject to $w_i x_i \leq m$ and maximize $P_i x_i$.*

PSEUDOCODE:

```
KNAPSACK(W, wt[], val[], n)
    create table K[0..n][0..W]

    for i ← 0 to n do
        for w ← 0 to W do
            if i == 0 OR w == 0 then
                K[i][w] ← 0

            else if wt[i-1] ≤ w then
                K[i][w] ← max(
                    val[i-1] + K[i-1][w -
wt[i-1]],
                    K[i-1][w]
                )

            else
                K[i][w] ← K[i-1][w]

    return K[n][W]
```

CODE-

```
1 #include <stdio.h>
2
3 int max(int a, int b) {
4     return (a > b) ? a : b;
5 }
6 int main() {
7     int n, W;
8
9     printf("Enter number of items: ");
10    scanf("%d", &n);
11
12    int val[n], wt[n];
13
14    printf("Enter values of %d items: ", n);
15    for (int i = 0; i < n; i++)
16        scanf("%d", &val[i]);
17
18    printf("Enter weights of %d items: ", n);
19    for (int i = 0; i < n; i++)
20        scanf("%d", &wt[i]);
21
22    printf("Enter maximum weight (capacity): ");
23    scanf("%d", &W);
24
25    int K[n + 1][W + 1];
26
27    for (int i = 0; i <= n; i++) {
28        for (int w = 0; w <= W; w++) {
29            if (i == 0 || w == 0)
30                K[i][w] = 0;
31            else if (wt[i - 1] <= w)
32                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
33            else
34                K[i][w] = K[i - 1][w];
35        }
36    }
37
38    printf("Maximum profit = %d\n", K[n][W]);
39
40    printf("Selected items: ");
41    int w = W;
42    for (int i = n; i > 0; i--) {
43        if (K[i][w] != K[i - 1][w]) {
44            printf("%d ", i);
45            w -= wt[i - 1];
46        }
47    }
48    printf("\n");
49
50    return 0;
```

OUTPUT-

```
Enter number of items: 3
Enter values of 3 items: 60 100 120
Enter weights of 3 items: 10 20 30
Enter maximum weight (capacity): 30
Maximum profit = 160
Selected items: 2 1
```

```
=== Code Execution Successful ===
```


Complexity:

Case	Time Complexity	Space Complexity
Best	$O(n^2)$	$O(n)$
Average	$O(n^2)$	$O(n)$
Worst	$O(n^2)$	$O(n)$

PROGRAM-04

Aim: Develop a program that calculates the most efficient way to multiply a chain of matrices by determining the optimal parenthesizing using dynamic programming.

Matrix chain multiplication:

- 1) *Matrix Chain Multiplication is a problem of finding the most efficient way to multiply a chain of matrices.*
- 2) *The goal is to minimize the total number of scalar multiplications.*
- 3) *Matrix multiplication is associative property:*
$$(A \times B) \times C = A \times (B \times C)$$
- 4) *Number of operations is different in each case.*
- 5) *It uses Dynamic Programming approach.*

PSEUDOCODE:

```
MATRIX_CHAIN_MULTIPLICATION(p, n)
    // p[] is dimension array of size n+1

    create tables m[1..n][1..n] and s[1..n][1..n]

    for i ← 1 to n do
        m[i][i] ← 0

    for L ← 2 to n do
        for i ← 1 to n-L+1 do
            j ← i + L - 1
            m[i][j] ← ∞

            for k ← i to j-1 do
                q ← m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]

                if q < m[i][j] then
                    m[i][j] ← q
                    s[i][j] ← k

    print "Minimum Cost = ", m[1][n]

// Function to print optimal parenthesization
PRINT_OPTIMAL(1, n)
```

```

PRINT_OPTIMAL(i, j)
    if i == j then
        print "A"i
    else
        print "("
        PRINT_OPTIMAL(i, s[i][j])
        PRINT_OPTIMAL(s[i][j] + 1, j)
        print ")"

```

CODE:

```

1  #include <stdio.h>
2  #include <limits.h>
3
4  int m[20][20];
5  int s[20][20];
6
7  void printOptimal(int i, int j) {
8      if (i == j) {
9          printf("A%d", i);
10     } else {
11         printf("(");
12         printOptimal(i, s[i][j]);
13         printOptimal(s[i][j] + 1, j);
14         printf(")");
15     }
16 }
17
18 int main() {
19     int n, i, j, k, L;
20     printf("Enter number of matrices: ");
21     scanf("%d", &n);
22
23     int p[n + 1];
24     printf("Enter the dimensions array (size %d):\n", n + 1);
25     for (i = 0; i <= n; i++) {
26         scanf("%d", &p[i]);
27     }
28     for (i = 1; i <= n; i++) {
29         m[i][i] = 0;
30     }
31     for (L = 2; L <= n; L++) {
32         for (i = 1; i <= n - L + 1; i++) {
33             j = i + L - 1;
34             m[i][j] = INT_MAX;
35             for (k = i; k < j; k++) {
36                 int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
37                 if (q < m[i][j]) {
38                     m[i][j] = q;
39                     s[i][j] = k;
40                 }
41             }
42         }
43     }
44     printf("\nMinimum number of multiplications: %d\n", m[1][n]);
45     printf("Optimal Parenthesization: ");
46     printOptimal(1, n);
47     printf("\n");
48     return 0;
49 }

```

OUTPUT-

```
Enter number of matrices: 3
Enter the dimensions array (size 4):
10 20 30 40

Minimum number of multiplications: 18000
Optimal Parenthesization: ((A1A2)A3)

=== Code Execution Successful ===
```

Complexity:

Case	Time Complexity	Space Complexity
Best	$O(n^3)$	$O(n^2)$
Average	$O(n^3)$	$O(n^2)$
Worst	$O(n^3)$	$O(n^2)$

PROGRAM-05

Aim: Develop a dynamic programming solution to find and display the longest common subsequence of two given strings.

Dynamic Programming:

- 1.) LCS is the longest sequence of characters that appears in both strings in the same order.*
- 2.) Characters must follow the same order, but they don't need to be adjacent.*
- 3.) Use a 2D table (dp array) to store results of smaller subproblems and build the solution step-by-step.*
- 4.) If characters match – add 1 to the result,
If not match – take minimum of left or top value in the table.*
- 5.) The final answer (length of LCS) is stored in the last cell of the table, and backtracking gives the actual subsequence.*

PSEUDOCODE:

```
LCS (X, Y)
    m ← length (X)
    n ← length (Y)

    create table L[0..m][0..n]

    for i = 0 to m:
        L[i][0] = 0
    for j = 0 to n:
        L[0][j] = 0

    for i = 1 to m:
        for j = 1 to n:
            if X[i-1] == Y[j-1]:
                L[i][j] = L[i-1][j-1] + 1
            else:
                L[i][j] = max(L[i-1][j], L[i][j-1])

    return L[m][n]
```

CODE-

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int max(int a, int b) {
5     return (a > b) ? a : b;
6 }
7
8 int main() {
9     char X[100], Y[100];
10     int m, n;
11
12     printf("Enter first string: ");
13     scanf("%s", X);
14
15     printf("Enter second string: ");
16     scanf("%s", Y);
17
18     m = strlen(X);
19     n = strlen(Y);
20
21     int L[m+1][n+1];
22
23     for (int i = 0; i <= m; i++) {
24         for (int j = 0; j <= n; j++) {
25             if (i == 0 || j == 0)
26                 L[i][j] = 0;
27             else if (X[i-1] == Y[j-1])
28                 L[i][j] = L[i-1][j-1] + 1;
29             else
30                 L[i][j] = max(L[i-1][j], L[i][j-1]);
31         }
32     }
33     printf("Length of LCS: %d\n", L[m][n]);
34
35     int i = m, j = n;
36     char lcs[100];
37     int index = L[m][n];
38     lcs[index] = '\0';
39
40     while (i > 0 && j > 0) {
41         if (X[i-1] == Y[j-1]) {
42             lcs[index-1] = X[i-1];
43             i--; j--; index--;
44         } else if (L[i-1][j] > L[i][j-1])
45             i--;
46         else
47             j--;
48     }
49
50     printf("LCS: %s\n", lcs);
51     return 0;
52 }
```

OUTPUT-

```
Enter first string: abcd
Enter second string: abce
Length of LCS: 3
LCS: abc
```

=== Code Execution Successful ===

Time Complexity: $O(m*n)$

Space Complexity: $O(m*n)$

PROGRAM-06

Aim: Implement the Traveling Salesman Problem to find the minimum-cost tour visiting all cities once.

Travelling Salesman Problem:

- 1.) The Travelling Salesman Problem (TSP) is a problem in which a salesman has to visit all cities and return to the starting city.*
- 2.) Each city must be visited exactly once, except the starting city which is visited again at the end.*
- 3.) Every route has a cost or distance, and we need to consider all possible paths.*
- 4.) The goal is to find the route with the minimum total cost, using methods like brute force, dynamic programming, or greedy approaches.*
- 5.) The final solution is the shortest possible route that covers all cities and returns to the starting point.*

PSEUDOCODE:

```
TSP(mask, pos)
    if mask == all cities visited:
        return cost[pos][0]

    if dp[mask][pos] != -1:
        return dp[mask][pos]

    ans = ∞

    for city = 0 to n-1:
        if city not visited in mask:
            newAns = cost[pos][city] +
                    TSP(mask | (1 << city), city)
            ans = min(ans, newAns)

    return dp[mask][pos] = ans
```


CODE-

```
1 #include <stdio.h>
2 #include <limits.h>
3 #include <string.h>
4
5 #define MAX 20
6
7 int n;
8 int cost[MAX][MAX];
9 int dp[1 << MAX][MAX];
10
11 int tsp(int mask, int pos) {
12     if (mask == (1 << n) - 1)
13         return cost[pos][0];
14
15     if (dp[mask][pos] != -1)
16         return dp[mask][pos];
17
18     int ans = INT_MAX;
19     for (int city = 0; city < n; city++) {
20         if ((mask & (1 << city)) == 0) {
21             int newAns = cost[pos][city] +
22                 tsp(mask | (1 << city), city);
23             if (newAns < ans)
24                 ans = newAns;
25         }
26     }
27     return dp[mask][pos] = ans;
28 }
29
30 int main() {
31     printf("Enter number of cities: ");
32     scanf("%d", &n);
33
34     printf("Enter cost matrix:\n");
35     for (int i = 0; i < n; i++) {
36         for (int j = 0; j < n; j++) {
37             scanf("%d", &cost[i][j]);
38         }
39     }
40
41     memset(dp, -1, sizeof(dp));
42     int result = tsp(1, 0);
43     printf("Minimum travelling cost: %d\n", result);
44
45     return 0;
46 }
```

OUTPUT-

```
Enter number of cities: 4
Enter cost matrix:
0 10 6 12
5 6 7 8
10 18 12 14
1 2 3 4
Minimum travelling cost: 27

=== Code Execution Successful ===
```

Complexity:

Time Complexity: $O(n^2 * 2^n)$

Space Complexity: $O(n * 2^n)$

PROGRAM-07

Aim: Create a program that uses BSF to systematically traverse all vertices of a graph starting from a given node.

Breadth First Search:

- 1.) BFS explores a graph or tree layer by layer, visiting every node at the current depth before moving to the next level down.*
- 2.) It uses a queue data structure to manage nodes, the first node discovered is the first one processed, ensuring the search expands uniformly.*
- 3.) In unweighted graphs, BFS is the standard for finding the shortest path because it evaluates all possibilities at distance N before looking at anything at distance $N+1$.*
- 4.) The algorithm tracks visited nodes using a set or Boolean array to avoid cycles and prevent the search from getting stuck in an infinite loop.*

PSEUDOCODE:

```
BFS(Graph, start)
    create queue Q
    mark all nodes as not visited

    mark start as visited
    enqueue start into Q

    while Q is not empty:
        node = dequeue Q
        print node

        for each neighbor of node:
            if neighbor not visited:
                mark neighbor visited
                enqueue neighbor
```

CODE-

```
1  #include <stdio.h>
2
3  #define MAX 20
4
5  int queue[MAX], front = -1, rear = -1;
6  int visited[MAX];
7  int graph[MAX][MAX];
8  int n;
9
10 void enqueue(int v) {
11     if (rear == MAX - 1)
12         return;
13     if (front == -1)
14         front = 0;
15     queue[++rear] = v;
16 }
17 int dequeue() {
18     return queue[front++];
19 }
20 int isEmpty() {
21     return (front > rear);
22 }
23
24 void bfs(int start) {
25     for (int i = 0; i < n; i++)
26         visited[i] = 0;
27
28     enqueue(start);
29     visited[start] = 1;
30
31     printf("BFS Traversal: ");
32
33     while (!isEmpty()) {
34         int node = dequeue();
35         printf("%d ", node);
36
37         for (int i = 0; i < n; i++) {
38             if (graph[node][i] && !visited[i]) {
39                 enqueue(i);
40                 visited[i] = 1;
41             }
42         }
43     }
44 }
45
46 int main() {
47     int start;
48
49     printf("Enter number of vertices: ");
50     scanf("%d", &n);
51
52     printf("Enter adjacency matrix:\n");
53     for (int i = 0; i < n; i++) {
54         for (int j = 0; j < n; j++) {
55             scanf("%d", &graph[i][j]);
56         }
57     }
58
59     printf("Enter starting vertex: ");
60     scanf("%d", &start);
61
62     bfs(start);
63
64     return 0;
65 }
```

OUTPUT-

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 0
0 1 0 0
Enter starting vertex: 0
BFS Traversal: 0 1 2 3

=== Code Execution Successful ===
```

Complexity:

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

PROGRAM-08

Aim: write a program to explore all nodes of a graph using Depth First Search (DFS) starting from a given node.

Depth First Search:

- 1.) DFS explores a graph by diving as deep as possible along a single branch before backtracking to explore other paths, moving vertically rather than horizontally.*
- 2.) The algorithm uses a stack data structure.*
- 3.) When a algorithm reaches a dead end, it backtracks to the previous node to check for remaining unexplored branches.*
- 4.) DFS typically requires less memory than BFS in wide graphs because it only needs to store the nodes along the current path from the root to the leaf.*

PSEUDOCODE:

```
DFS(Graph, node)
    mark node as visited
    print node

    for each neighbor of node:
        if neighbor not visited:
            DFS(Graph, neighbor)
```

CODE-

```
1  #include <stdio.h>
2
3  #define MAX 20
4
5  int graph[MAX][MAX];
6  int visited[MAX];
7  int n;
8
9  void dfs(int node) {
10     printf("%d ", node);
11     visited[node] = 1;
12
13     for (int i = 0; i < n; i++) {
14         if (graph[node][i] && !visited[i]) {
15             dfs(i);
16         }
17     }
18 }
19
20 int main() {
21     int start;
22     printf("Enter number of vertices: ");
23     scanf("%d", &n);
24
25     printf("Enter adjacency matrix:\n");
26     for (int i = 0; i < n; i++) {
27         for (int j = 0; j < n; j++) {
28             scanf("%d", &graph[i][j]);
29         }
30     }
31     for (int i = 0; i < n; i++)
32         visited[i] = 0;
33
34     printf("Enter starting vertex: ");
35     scanf("%d", &start);
36
37     printf("DFS Traversal: ");
38     dfs(start);
39
40     return 0;
}
```

OUTPUT-

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 0
0 1 0 0
Enter starting vertex: 0
DFS Traversal: 0 1 3 2

=== Code Execution Successful ===
```

Complexity:

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$

